Poids Mort Numérique

L'assistant de sobriété numérique personnelle

Livrable 2 — Implémentation, Analyse et Évaluation

Mini-projet TI616 « Numérique Durable »

EFREI Paris — 2025-2026

Coordinateur : Yvan Guifo

Équipe :

Antoine Poirier — Noé Millereux — Guillaume Ortells

Dépôt : github.com/Itohax78/poids-mort-numerique

Sommaire

1. Présentation du projet

1.1 Rappel de la proposition de valeur

Poids Mort Numérique est un outil personnel d'inventaire et de nettoyage des services en ligne inutilisés. Chaque utilisateur déclare ses comptes, abonnements, espaces cloud et newsletters, leur affecte un statut (actif, dormant, supprimé) et voit progresser un score de sobriété au fur et à mesure du nettoyage. Le site estime également l'énergie évitée (kWh/an) et les émissions de CO_2 évitées (g/an), pour rendre concrète une démarche habituellement abstraite.

L'idée force du projet : un outil qui aide à réduire l'empreinte numérique se doit d'être lui-même exemplaire. Cette cohérence entre fond et forme est le fil conducteur de toutes les décisions techniques.

1.2 Périmètre du MVP

Fonctionnalités retenues pour cette première version, validées en Partie 1 :

- Inscription, connexion, déconnexion (sessions Flask signées).
- CRUD complet du profil utilisateur (création, lecture, modification, suppression).
- CRUD complet des services numériques.
- Changement de statut actif → dormant → supprimé.
- Calcul et affichage d'un score de sobriété + estimation kWh / CO_2 évités.
- Tableau de bord personnel avec récapitulatif.
- Page d'accueil présentant le service et ses engagements écologiques.
- Espace administrateur : statistiques globales, gestion des comptes.

Fonctionnalités explicitement reportées en V2 : export CSV, classement anonyme, indicateur de poids de page, suggestions automatiques de nettoyage, mode sombre natif.

1.3 Utilisateurs cibles

Étudiants et jeunes actifs sensibilisés (ou à sensibiliser) à l'impact environnemental du numérique. Plus largement, tout internaute souhaitant reprendre le contrôle de sa présence en ligne. Trois rôles dans le système : visiteur (lecture page d'accueil + inscription), utilisateur (CRUD sur ses données), admin (statistiques globales + gestion des comptes).

1.4 Contraintes Green IT retenues

Six engagements pris dès le cadrage et tenus tout au long de l'implémentation :

- Aucun framework front-end. HTML/CSS vanilla, polices système, zéro requête réseau externe.
- Aucun ORM. Accès BDD direct via le module sqlite3 de la stdlib Python.
- Aucune image décorative ni police externe.
- Pagination obligatoire (LIMIT/OFFSET) sur toutes les listes.
- Catégories prédéfinies plutôt que saisie libre, pour éviter les doublons et les données non structurées.
- Données utilisateur strictement minimales : pseudo, email, mot de passe hashé. Pas de prénom, pas de photo, pas de téléphone.

2. Architecture et conception

2.1 Architecture générale

Le système suit une architecture MVC classique, en monolithe Python.

Trois couches :

- Présentation : templates HTML rendus côté serveur par Jinja2. CSS unique (4 Ko). JavaScript minimal, uniquement pour les confirmations natives via window.confirm().
- Logique applicative : Flask. Un seul Blueprint, une douzaine de routes, gestion des sessions, validation des entrées, calcul du score et de l'impact évité.
- Données : SQLite (fichier unique). 3 tables, 2 index. Aucun ORM.

Les interactions sont simples et linéaires : navigateur → requête HTTP → Flask traite la logique → interroge SQLite → renvoie la page HTML complète. Pas d'API REST séparée, pas de WebSocket, pas de cache externe (Redis, Memcached). Un seul processus Python suffit pour servir l'application.

2.2 Schéma de la base de données

Trois tables, modèle minimal cohérent avec le cadrage du Livrable 1 :

Table	Champs	Notes
users	id (PK), pseudo (UNIQUE), email (UNIQUE), mot_de_passe_hash, role, date_inscription	Pas de nom/prénom/téléphone/photo. Le pseudo seul suffit à l'identification.
services	id (PK), nom, statut, date_ajout, date_derniere_utilisation, user_id (FK), categorie_id (FK)	ON DELETE CASCADE depuis users. Index sur user_id et statut uniquement.
categories	id (PK), nom (UNIQUE), impact_kwh_an	Pré-remplie : 6 catégories. Évite la saisie libre.

Relations : users 1—N services ; categories 1—N services. Pas de relation entre utilisateurs.

2.3 Diagrammes UML

Les trois diagrammes UML demandés (cas d'utilisation, classes/MLD, séquence) ont été produits avec PlantUML et sont déposés dans **docs/diagrammes/** du dépôt. Description textuelle ci-dessous, une lecture des fichiers **.puml** donne le rendu graphique.

Diagramme de cas d'utilisation

Trois acteurs : Visiteur, Utilisateur, Admin. Visiteur peut consulter la page d'accueil et créer un compte. Utilisateur hérite de Visiteur et y ajoute : se connecter, gérer son profil, ajouter/modifier/supprimer ses services, changer leur statut, consulter son tableau de bord et son score, se déconnecter. Admin hérite d'Utilisateur et y ajoute : accéder aux statistiques globales et supprimer n'importe quel compte.

Diagramme de classes / MLD

Cf. tableau §2.2 ci-dessus. Cardinalités : User 1—N Service ; Category 1—N Service.

Diagramme de séquence — Ajout d'un service numérique

- L'utilisateur connecté remplit le formulaire (nom, catégorie, statut).
- Le navigateur envoie POST /services/ajouter avec les champs.
- Flask vérifie la session (décorateur login_required) puis valide les entrées.

- models.create_service exécute INSERT INTO services avec une requête paramétrée.
- Flash success « Service ajouté ».
- Redirection 302 vers /dashboard, qui recalcule le score via models.get_user_stats.
- Le navigateur affiche le tableau de bord avec score et impact mis à jour.

3. Choix technologiques justifiés

Chaque choix technique a été arbitré selon trois critères : (a) répond-il vraiment au besoin ? (b) son coût en ressources est-il proportionné ? (c) restera-t-il maintenable par 4 étudiants en quelques semaines ?

Couche	Technologie	Justification Green IT
Front-end	HTML5 + CSS3 + JS minimal	Zéro framework, zéro bundler, zéro étape de build. Page d'accueil en 2,4 Ko HTML + 4 Ko CSS minifié.
Polices	system-ui, -apple-system, sans-serif	Aucune police externe : zéro requête réseau supplémentaire, zéro flash de texte, zéro Ko téléchargé.
Back-end	Flask 3 (Python 3.10+)	Micro-framework de ~1 Mo. Inclut Jinja2 et Werkzeug. Pas de magie cachée, pas de système de plugins.
ORM	Aucun (sqlite3 stdlib)	Évite ~200 Mo de dépendances (SQLAlchemy + tooling). Requêtes paramétrées explicites = code plus clair et plus sûr.
Base de données	SQLite	Fichier unique, zéro serveur, zéro réseau, zéro RAM au repos. Largement suffisant pour le volume attendu.
Sécurité	Werkzeug security (inclus)	Hashage pbkdf2-sha256 des mots de passe. Aucune dépendance ajoutée.
Production	Gunicorn	Serveur WSGI Unix de ~600 Ko, 1 processus, peu de RAM. La seule dépendance ajoutée hors Flask.
Versionnement	Git + GitHub	Imposé par le sujet. Sert aussi pour Issues, Projects et PR.
Hébergement	Render (free tier)	Free tier généreux, hébergement européen. Vérification : The Green Web Foundation indique un hébergeur partiellement renouvelable.
Documentation	Markdown dans le dépôt	Versionné avec le code, lisible sans outil tiers.

3.1 Pourquoi pas React, Vue ou Angular ?

Un framework JS impose un bundler (Webpack, Vite), un processus de build, un serveur de développement, et génère un bundle de 100 Ko minimum (React 18 + ReactDOM = ~140 Ko gzippés) avant la moindre ligne de code utile. Pour notre interface (formulaires, listes, barres de progression CSS), le rendu côté serveur Jinja2 suffit : zéro JS téléchargé par défaut, zéro étape de compilation, zéro dépendance npm. Le gain en sobriété est immédiat et la maintenance simplifiée.

3.2 Pourquoi pas PostgreSQL ou MySQL ?

Ces SGBD nécessitent un serveur séparé, de la configuration réseau, et consomment de la mémoire en permanence. SQLite stocke tout dans un fichier unique, ne consomme aucune ressource au repos, supporte largement le volume attendu (quelques milliers de lignes au maximum) et simplifie le déploiement. Si le projet grandissait au-delà de quelques centaines d'utilisateurs concurrents, PostgreSQL deviendrait pertinent — mais pas avant.

3.3 Pourquoi pas un ORM (SQLAlchemy / Peewee) ?

Pour 3 tables et une dizaine de requêtes, l'ORM ajoute plus de complexité qu'il n'en retire. SQLAlchemy seul représente plusieurs dizaines de Mo de dépendances. L'écriture directe en SQL reste lisible (chaque requête fait au plus 5 lignes), évite le risque des requêtes N+1 et donne un contrôle exact sur les colonnes lues.

4. Implémentation

4.1 Vue d'ensemble du code

Le code source tient en un peu plus de 600 lignes Python et 300 lignes de templates :

Fichier	Lignes	Rôle
app/app.py	~70	Factory Flask, hook de compression gzip, config sécurité.
app/models.py	~270	Accès BDD : init, CRUD users, CRUD services, statistiques.
app/routes.py	~280	Blueprint principal : routes, validation, autorisation.
app/templates/*	~310	9 templates Jinja2 héritant tous de base.html.
app/static/css/*	~190	Une feuille (4 Ko minifiée), aucune image.
database/schema.sql	~55	3 tables, 2 index, 6 catégories pré-remplies.
database/seed.py	~40	Données de démonstration.
tests/tests.py	~190	12 scénarios fonctionnels et de sécurité.

4.2 CRUD utilisateurs

Les 4 opérations sont réparties entre routes.py (validation, autorisation) et models.py (SQL paramétré). Extrait représentatif de la création d'utilisateur :

```
def create_user(pseudo, email, mot_de_passe, role="user"):
    pwd_hash = generate_password_hash(mot_de_passe)
    try:
        with get_db() as conn:
            cur = conn.execute(
                "INSERT INTO users (pseudo, email, mot_de_passe_hash, role) "
                "VALUES (?, ?, ?, ?)",
                (pseudo, email, pwd_hash, role),
            )
            conn.commit()
            return cur.lastrowid
    except sqlite3.IntegrityError:
        return None  # pseudo ou email déjà pris
```

Trois éléments à noter : (1) requête paramétrée (?) qui empêche toute injection SQL ; (2) hashage immédiat du mot de passe via Werkzeug (pbkdf2-sha256) ; (3) gestion explicite du conflit d'unicité.

4.3 CRUD services numériques

Toutes les opérations vérifient l'appartenance du service à l'utilisateur connecté. Exemple : la fonction get_service prend un user_id optionnel et l'ajoute à la clause WHERE, ce qui rend impossible la lecture d'un service d'autrui même via une URL forgée. La liste des services est paginée (LIMIT/OFFSET) avec un défaut de 20 résultats par page, comme imposé par les contraintes Green IT du sujet.

4.4 Calcul du score de sobriété

Le score est calculé en une seule requête SQL (groupement par statut) puis traité en Python. Logique : un service dormant compte pour 0,5 « nettoyé », un service supprimé pour 1, un actif pour 0. Score = 100 × (somme des nettoyés) / (nombre total de services). Si l'utilisateur n'a pas encore de service, score = 100 (rien à nettoyer).

L'impact évité est calculé à partir des kWh annuels de chaque catégorie (barème intégré, source ADEME / Collectif GreenIT.fr) : impact total des services supprimés + 50 % de celui des dormants. La conversion en CO_2 utilise le facteur d'émission moyen de l'électricité française (~50 g CO_2eq/kWh).

4.5 Optimisations Green IT intégrées

Trois optimisations ont été implémentées et mesurées :

- Minification CSS — passage de 5 821 à 4 042 octets (-31 %).
- Compression gzip à la volée via un hook after_request, sans dépendance externe (gzip de la stdlib).
- Cache HTTP statique de 24 heures (Cache-Control: max-age=86400) sur le CSS.

Détails et mesures comparées en §5 ci-dessous.

5. Analyse de l'empreinte carbone

5.1 Outils utilisés

Conformément à la Phase 5.1 du sujet, plusieurs outils ont été mobilisés :

- Mesures internes : test client Flask qui enregistre la taille exacte des réponses HTTP.
- Website Carbon Calculator (websitecarbon.com) — empreinte CO_2 par visite (déploiement).
- EcoIndex (ecoindex.fr) — note A à G basée sur poids, requêtes, complexité DOM.
- Google Lighthouse (DevTools Chrome) — Performance, Accessibilité, Bonnes pratiques.
- PageSpeed Insights — Core Web Vitals en conditions réelles.

Les captures d'écran complètes des outils en ligne sont déposées dans docs/eco-mesures/ après déploiement final.

5.2 Mesure initiale (avant optimisations)

Mesures effectuées avec un test client Flask sur les 3 pages principales, CSS non minifié, sans gzip :

Page	HTML	HTML + CSS	Requêtes HTTP
Accueil	2 403 octets	8 224 octets	2 (HTML + CSS)
Connexion	1 447 octets	7 268 octets	2
Tableau de bord	7 474 octets	13 295 octets	2

À ce stade, les résultats étaient déjà excellents grâce aux choix architecturaux (pas de framework, pas de média), mais trois marges de progression ont été identifiées : commentaires et espaces dans le CSS, absence de compression à la volée, absence d'en-têtes de cache.

5.3 Sources de pollution identifiées

Source potentielle	Présent ?	Décision
Images non compressées	Non	Aucune image dans le projet.
Scripts JS inutilisés	Non	Aucun JS externe ; uniquement quelques confirm().
Polices externes	Non	Polices système uniquement.
Requêtes redondantes	Limité	1 seule requête CSS par session ; HTML inévitable.
Dépendances surdimensionnées	Non	Flask + Gunicorn = 2 dépendances directes.
CSS non minifié	Oui	→ optimisation 1.
Pas de compression HTTP	Oui	→ optimisation 2.
Pas de cache HTTP statique	Oui	→ optimisation 3.

5.4 Optimisations appliquées

Trois optimisations concrètes, codées et mesurées :

Optimisation 1 — Minification CSS

Suppression manuelle des commentaires, espaces non significatifs et sauts de ligne. Génération de style.min.css référencé dans base.html.

Résultat : 5 821 → 4 042 octets bruts (-31 %). Avec gzip ensuite : 4 042 → 1 308 octets (-78 % vs CSS d'origine).

Optimisation 2 — Compression gzip à la volée

Ajout d'un hook after_request dans la factory Flask qui compresse les réponses textuelles (HTML, CSS) lorsque le client envoie Accept-Encoding: gzip. Implémentation en 25 lignes via gzip.GzipFile de la bibliothèque standard, donc aucune dépendance externe ajoutée.

Le hook ignore les réponses < 500 octets (overhead non rentable) et celles déjà compressées. Sur le déploiement Render, le reverse proxy ferait la même chose au-dessus, mais l'avoir au niveau applicatif garantit la compression aussi en local et derrière un Gunicorn nu.

Optimisation 3 — Cache HTTP statique

Configuration SEND_FILE_MAX_AGE_DEFAULT = 86400 (24 heures). À partir de la 2e page consultée dans la même session, le navigateur ne re-télécharge pas le CSS, ce qui ramène le coût d'une navigation interne au seul HTML (souvent < 1 Ko gzippé).

5.5 Comparaison avant / après

Mesures sur les 3 pages principales, avec CSS minifié, gzip activé, première visite (cache vide) :

Indicateur	Avant	Après	Gain
Page accueil — total transféré	8 224 octets	2 479 octets	-70 %
Page connexion — total transféré	7 268 octets	2 063 octets	-72 %
Tableau de bord — total	13 295 octets	2 781 octets	-79 %
CSS seul (brut → gzip)	5 821 octets	1 308 octets	-78 %
Nombre de requêtes HTTP / page	2	2 (1 après cache)	-50 % dès la 2e page
Dépendances Python directes	2 (Flask, Gunicorn)	2 (idem)	0 (objectif tenu)
JS chargé	0 octet	0 octet	0 (rien à optimiser)
Images chargées	0	0	0 (idem)

Gain moyen sur le poids total transféré : -74 %.

5.6 Estimations indicatives via les outils en ligne

Une fois le site déployé, les valeurs cibles attendues d'après la littérature Green IT et les outils (EcoIndex, Website Carbon) sont les suivantes (à confirmer en production) :

Outil / Indicateur	Valeur attendue	Comparaison
EcoIndex	Note A (~95/100)	Site "léger" classique : C/D
Website Carbon — CO_2 / visite	0,01-0,03 g CO_2	Page web moyenne (2024) : ~0,8 g
Lighthouse Performance	95-100 / 100	Bon site classique : 70-85
Lighthouse Accessibilité	95-100 / 100	—
FCP (First Contentful Paint)	< 0,5 s (3G simulé)	Cible Google : < 1,8 s
LCP (Largest Contentful Paint)	< 0,8 s	Cible Google : < 2,5 s

Les valeurs exactes seront relevées avec captures d'écran lors de la séance de présentation finale (le site doit alors être en ligne). Le pré-déploiement local laisse présager un classement EcoIndex A compte tenu du poids (< 10 Ko transférés) et de la complexité DOM (~80 nœuds).

6. Tests et validation

6.1 Tests fonctionnels

12 scénarios automatisés ont été codés dans tests/tests.py via unittest (stdlib, aucune dépendance externe). Lancement :

```
python -m tests.tests
```

Tous les scénarios passent. Détail :

#	Scénario	Résultat attendu	Statut
01	Créer un utilisateur valide	Compte créé, redirection, message succès	OK
02	Créer un utilisateur (email vide)	Erreur côté serveur	OK
03	Modifier le pseudo connecté	Données mises à jour en BDD	OK
04	Supprimer son compte (avec confirmation)	Compte supprimé, services en cascade	OK
05	Lister les utilisateurs (admin)	Affichage paginé correct	OK
06	Créer un service numérique	Service inséré, visible sur dashboard	OK
07	Connexion identifiants valides	Redirection vers dashboard	OK
08	Connexion mauvais mot de passe	Message d'erreur, pas de session	OK
09	Accès dashboard non connecté	Redirection 302 vers /connexion	OK
10	Mots de passe hashés en BDD	Pas de clair, préfixe pbkdf2:	OK
11	Tentative d'injection SQL ' OR 1=1 --	Repoussée, identifiants incorrects	OK
12	Accès /admin par user normal	HTTP 403 Forbidden	OK

6.2 Tests de performance

Les tests Lighthouse seront exécutés sur le déploiement final pour les 3 pages principales (accueil, connexion, dashboard). Captures déposées dans docs/eco-mesures/.

Métriques cibles attendues, conformes aux contraintes du sujet :

- Score Performance > 80/100 — visé : > 95.
- Poids total < 500 Ko par page — mesuré : < 10 Ko (cache vide).
- Nombre de requêtes HTTP < 15 par page — mesuré : 2.
- FCP < 1,8 s — visé : < 0,5 s même en 3G simulé.
- LCP < 2,5 s — visé : < 1,0 s.

6.3 Tests de sécurité

Les 4 vérifications imposées par la Phase 6.3 du sujet ont été effectuées :

Vérification	Méthode	Résultat
Aucun mot de passe en clair en BDD	SELECT direct sur users + test automatisé n°10	OK — préfixe pbkdf2-sha256: vérifié

Vérification	Méthode	Résultat
Aucune variable sensible dans le dépôt	git log + grep sur password / secret + .env dans .gitignore	OK — .env.example fourni, .env exclu
Formulaires rejettent les entrées malformées	Test n°11 (injection SQL ' OR 1=1) + validations côté serveur	OK — requêtes paramétrées partout
Page protégée non accessible par URL forgée	Test n°09 (GET /dashboard sans session)	OK — redirection 302 vers /connexion ; test n°12 : 403 sur /admin

7. Organisation de l'équipe et collaboration

7.1 Répartition des tâches

Membre	Périmètre principal	Branches Git associées
Antoine Poirier	Modélisation BDD (schema.sql), couche models.py, calcul du score de sobriété	feature/database, feature/score
Noé Millereux	Templates Jinja2, CSS sobre, ergonomie, wireframes	feature/homepage, feature/dashboard
Guillaume Ortells	Authentification, espace admin, tests fonctionnels et de sécurité	feature/auth, feature/admin, feature/tests

7.2 Workflow Git

Trois niveaux de branches :

- main — branche stable, protégée, déployée. Aucune écriture directe.
- dev — branche d'intégration. Toutes les features y sont mergées avant main.
- feature/* — une branche par fonctionnalité, ouverte depuis dev, refermée par PR.

Chaque PR fait référence à une issue GitHub correspondante et doit être relue par un autre membre avant merge. Les PR triviales (typos, doc) peuvent être self-mergées avec un commentaire justificatif.

7.3 Suivi des tâches

Le backlog du Livrable 1 (US-01 à US-14) a été retranscrit en GitHub Issues, avec labels de priorité (haute / moyenne / basse) et assignees. Un GitHub Project (board Kanban) avec colonnes To Do / In Progress / In Review / Done permet de visualiser l'avancement. Les captures du board sont versionnées dans docs/eco-mesures/ et présentées à l'oral.

7.4 Conventions de commit

Convention Conventional Commits adaptée en français :

```
feat(auth)     : ajout du formulaire d'inscription
fix(services)  : correction du calcul du score quand 0 service
docs(readme)   : mise à jour de la section installation
refactor(db)   : extraction de la logique de stats
test(scenarios): ajout du scénario de connexion KO
```

8. Discussion et conclusion

8.1 Défis rencontrés

Le principal défi n'était pas technique mais doctrinal : à chaque fonctionnalité, il a fallu résister à la tentation d'ajouter "ce qui ferait pro" — un graphique en camembert, une animation au survol, une icône de service, un favicon SVG dégradé. Chacun de ces ajouts pris isolément ne pèse rien ; cumulés, ils produiraient le site classique de 500 Ko que le sujet nous invite précisément à éviter.

Sur le plan technique, le seul point délicat a été le hook gzip dans Flask : par défaut, send_file utilise le mode passthrough qui interdit la lecture des données. La correction (response.direct_passthrough = False) est documentée dans le code.

8.2 Compromis entre fonctionnalités et sobriété

Trois arbitrages explicites ont été faits :

- Estimation kWh/CO_2 via barème intégré plutôt qu'API externe. Précision moindre, mais zéro requête réseau, zéro dépendance, et la promesse "site sobre" est respectée jusqu'au bout.
- Catégories prédéfinies plutôt que saisie libre. L'utilisateur perd un peu de souplesse, mais on évite les doublons ("Gmail" / "gmail" / "G-mail") et les données non structurées.
- Pas de graphique. Une simple barre CSS pour le score suffit. Recharts ou Chart.js pèsent 50 à 200 Ko pour afficher des informations qu'une barre HTML rend très bien.

8.3 Pistes d'amélioration (V2)

- Indicateur de poids de page en footer (déjà au backlog : US-13).
- Export CSV des données personnelles (US-14) — pour la portabilité RGPD.
- Suggestions automatiques : services actifs depuis > 12 mois sans modification → proposer dormant.
- Mode sombre natif via prefers-color-scheme (gain énergétique réel sur écrans OLED).
- Pipeline GitHub Actions : lint flake8 + tests + alerte si poids static > 50 Ko.

8.4 Regard critique

Notre démarche a un biais : il est facile de produire un "site Green" lorsqu'on choisit dès le départ un sujet qui se prête à la sobriété (un outil texte, sans média, sans temps réel). Le défi serait plus intéressant sur un service où l'utilisateur attend des images, des cartes, des vidéos. Cela dit, la discipline acquise — questionner chaque champ, chaque dépendance, chaque ressource — s'applique à tout projet.

L'autre limite honnête : nos chiffres d'énergie évitée sont des ordres de grandeur, pas des mesures. Une vraie comptabilité demanderait l'accès aux données d'opération des fournisseurs de services, ce qui n'est pas réaliste. Mais l'objectif n'est pas de produire un certificat ISO, c'est de rendre concret pour l'utilisateur le fait que ses comptes inutilisés ont un coût.

8.5 Conclusion

Poids Mort Numérique tient en moins de 1 100 lignes de code, deux dépendances Python, une feuille CSS de 4 Ko et une base de données de quelques kilo-octets. Il sert toutes les fonctionnalités attendues d'un petit service web (CRUD, authentification, autorisations, statistiques) avec un budget de transfert de moins de 10 Ko par page après optimisations, soit deux ordres de grandeur sous la moyenne du web actuel (2,3 Mo selon HTTP Archive 2024).

Le projet illustre une thèse simple : la sobriété numérique n'est pas une contrainte de pauvreté, c'est un choix de conception qui rend les sites plus rapides, plus accessibles, plus maintenables — et, accessoirement, plus écologiques. Le "plus moderne" n'est pas toujours le "plus pertinent", comme le rappelait le sujet ; nous l'avons éprouvé en pratique.

« La perfection est atteinte non pas lorsqu'il n'y a plus rien à ajouter, mais lorsqu'il n'y a plus rien à retirer. »

— Antoine de Saint-Exupéry

9. Annexes

9.1 Backlog final (issu du Livrable 1, état d'avancement)

ID	Fonctionnalité	Priorité	Statut
US-01	Inscription / Création de compte	Haute	Terminé
US-02	Connexion / Déconnexion	Haute	Terminé
US-03	Modification du profil utilisateur	Haute	Terminé
US-04	Suppression de compte	Moyenne	Terminé
US-05	Ajout d'un service numérique	Haute	Terminé
US-06	Modification / Suppression d'un service	Haute	Terminé
US-07	Changement de statut	Haute	Terminé
US-08	Calcul et affichage du score de sobriété	Haute	Terminé
US-09	Tableau de bord personnel	Haute	Terminé
US-10	Statistiques globales (admin)	Moyenne	Terminé
US-11	Page d'accueil avec engagements éco	Moyenne	Terminé
US-12	Affichage de l'impact évité (CO_2, kWh)	Moyenne	Terminé
US-13	Indicateur de poids de page	Basse	Reporté V2
US-14	Export CSV des données personnelles	Basse	Reporté V2

9.2 Récapitulatif des livrables présents au dépôt

#	Livrable	Emplacement
1	Code source front-end	app/templates/, app/static/css/
2	Code source back-end	app/app.py, app/models.py, app/routes.py
3	Schéma de base de données	database/schema.sql
4	Script de seed	database/seed.py
5	Fichier README.md complet	racine du dépôt
6	Rapport PDF (ce document)	docs/rapport.pdf
7	Diagrammes UML	docs/diagrammes/ (PlantUML)
8	Wireframes	docs/wireframes/
9	Captures d'écran outils éco	docs/eco-mesures/
10	Tableau de tests fonctionnels	§6.1 du présent rapport + tests/tests.py
11	Fichier .env.example	racine du dépôt
12	Fichier .gitignore	racine du dépôt

9.3 Commandes utiles

```
# Cloner et installer
```

```
git clone https://github.com/Itohax78/poids-mort-numerique.git
cd poids-mort-numerique
python -m venv venv && source venv/bin/activate
pip install -r requirements.txt

# Initialiser une base de démo
python -m database.seed

# Lancer en local
python -m app.app

# Lancer la suite de tests
python -m tests.tests

# Lancer en production (Gunicorn)
export FLASK_SECRET_KEY=$(python -c 'import secrets;print(secrets.token_hex(32))')
export FLASK_ENV=production
gunicorn -w 2 -b 0.0.0.0:8000 app.app:app
```

9.4 Sources et références

- Référentiel GR491 — Green IT (gr491.isit-europe.org)
- Collectif Green IT — greenit.fr
- Eco-conception web — eco-conception-web.com
- ADEME — Impact environnemental du numérique
- HTTP Archive — Web Almanac 2024 (httparchive.org/reports/state-of-the-web)
- Documentation Flask — flask.palletsprojects.com
- Documentation SQLite — sqlite.org/docs.html